In [12]:
```
import os
import pandas as pd
import matplotlib.pyplot as plt
import cv2
from tqdm.notebook import tqdm
import random
import numpy as np
from PIL import Image
import json

import torch
import torch.nn as nn
from torchvision import models
from torchsummary import summary
from torchvision import datasets, transforms
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader, ConcatDataset

import matplotlib.pyplot as plt
import matplotlib.font_manager as fm
```

Mission 1-1

In [2]:
```
# 파일명에서 특징을 추출하여 데이터프레임으로 정리하는 함수
def extract_features_from_directory(directory):
    data = {
        "File_Name": [],
        "W/T": [],
        "ImgaeID": [],
        "Era": [],
        "Style": [],
        "Gender": []
    }

    # 디렉토리 내 파일 탐색
    for filename in os.listdir(directory):
        if filename.endswith(".jpg"):  # 확장자가 jpg인 파일만 처리
            data["File_Name"].append(filename)
            parts = filename.replace(".jpg", "").split("_")

            data["W/T"].append(parts[0])
            data["ImgaeID"].append(parts[1])
            data["Era"].append('20'+parts[2] if int(parts[2])<50 else '19
            data["Style"].append(parts[3])
            data["Gender"].append("여성" if parts[4] == "W" else "남성")

    return pd.DataFrame(data)
```

1-1. 주어진 이미지 데이터의 파일 명은 아래와 같은 형식이다. **"{W/T}*{이미지ID}*{시대별}*{스타일별}*{성별}.jpg"**이에 기반하여 **"이미지ID"** 수 기준으로 **"성별 & 스타일"** 통계치를 아래 표 형식으로 기입한다.

Train Images 분석

In [3]:
```
train_image_dir = './data/training_image'
valid_image_dir = './data/validation_image'
```

In [4]:
```
train_df = extract_features_from_directory(train_image_dir)
train_df.describe()
```

Out[4]:

	File_Name	W/T	ImgaeID	Era	Style	Gender
count	4070	4070	4070	4070	4070	4070
unique	4070	2	4066	8	27	2
top	W_14444_80_bodyconscious_W.jpg	W	10028	2019	normcore	남성
freq	1	4009	2	639	517	2248

In [5]:
```
# 'Gender'와 'Style' 열을 선택하여 새로운 DataFrame 생성
gender_style_train_df = train_df[['Gender', 'Style']]

# 결과 출력
print(gender_style_train_df)
```

```
     Gender           Style
0        여성   bodyconscious
1        남성        normcore
2        여성           disco
3        여성  sportivecasual
4        여성        feminine
...     ...             ...
4065     남성        normcore
4066     여성        military
4067     남성          hippie
4068     여성            punk
4069     남성             ivy

[4070 rows x 2 columns]
```

In [6]:
```
# 성별(Gender)과 스타일(Style)로 그룹화하여 각 그룹의 개수를 계산
gender_style_counts = gender_style_train_df.groupby(['Gender', 'Style']).

# 결과 출력
print(gender_style_counts)
```

	Gender	Style	count
0	남성	bold	268
1	남성	hiphop	274
2	남성	hippie	260
3	남성	ivy	237
4	남성	metrosexual	278
5	남성	mods	269
6	남성	normcore	364
7	남성	sportivecasual	298
8	여성	athleisure	67
9	여성	bodyconscious	95
10	여성	cityglam	67
11	여성	classic	77
12	여성	disco	37
13	여성	ecology	64
14	여성	feminine	154
15	여성	genderless	77
16	여성	grunge	31
17	여성	hiphop	48
18	여성	hippie	91
19	여성	kitsch	91
20	여성	lingerie	55
21	여성	lounge	45
22	여성	military	33
23	여성	minimal	139
24	여성	normcore	153
25	여성	oriental	78
26	여성	popart	41
27	여성	powersuit	120
28	여성	punk	65
29	여성	space	37
30	여성	sportivecasual	157

In [7]:
```
# 저장할 디렉토리와 파일 경로 설정
output_dir = "./output"
csv_file_path = os.path.join(output_dir, "1_1_train_images.csv")

# 디렉토리가 없으면 생성
if not os.path.exists(output_dir):
    os.makedirs(output_dir)

# DataFrame을 csv 파일로 저장
gender_style_counts.to_csv(csv_file_path, index=False)

# 성공 메시지 출력
print('저장 성공!!\n경로 :', csv_file_path)
```

```
저장 성공!!
경로 : ./output/1_1_train_images.csv
```

Valid Images 분석

In [8]:
```
valid_df = extract_features_from_directory(valid_image_dir)
valid_df.describe()
```

Out[8]:

	File_Name	W/T	ImgaeID	Era	Style	Gender
count	951	951	951	951	951	951
unique	951	2	951	8	27	2
top	W_14444_80_bodyconscious_W.jpg	W	14444	1950	sportivecasual	남성
freq	1	941	1	145	100	525

In [9]:
```
# 'Gender'와 'Style' 열을 선택하여 새로운 DataFrame 생성
gender_style_valid_df = valid_df[['Gender', 'Style']]

# 결과 출력
print(gender_style_valid_df)
```

```
    Gender           Style
0       여성    bodyconscious
1       여성   sportivecasual
2       여성         feminine
3       여성       genderless
4       여성           kitsch
..     ...              ...
946     여성            punk
947     남성   sportivecasual
948     여성          minimal
949     남성           hiphop
950     남성              ivy

[951 rows x 2 columns]
```

In [10]:
```
# 성별(Gender)과 스타일(Style)로 그룹화하여 각 그룹의 개수를 계산
gender_style_counts = gender_style_valid_df.groupby(['Gender', 'Style']).

# 결과 출력
print(gender_style_counts)
```

	Gender	Style	count
0	남성	bold	57
1	남성	hiphop	66
2	남성	hippie	82
3	남성	ivy	79
4	남성	metrosexual	58
5	남성	mods	80
6	남성	normcore	51
7	남성	sportivecasual	52
8	여성	athleisure	14
9	여성	bodyconscious	23
10	여성	cityglam	18
11	여성	classic	22
12	여성	disco	10
13	여성	ecology	17
14	여성	feminine	44
15	여성	genderless	12
16	여성	grunge	10
17	여성	hiphop	8
18	여성	hippie	14
19	여성	kitsch	22
20	여성	lingerie	5
21	여성	lounge	8
22	여성	military	9
23	여성	minimal	35
24	여성	normcore	20
25	여성	oriental	18
26	여성	popart	8
27	여성	powersuit	34
28	여성	punk	12
29	여성	space	15
30	여성	sportivecasual	48

In [11]:
```python
# 저장할 디렉토리와 파일 경로 설정
output_dir = "./output"
csv_file_path = os.path.join(output_dir, "1_1_valid_images.csv")

# 디렉토리가 없으면 생성
if not os.path.exists(output_dir):
    os.makedirs(output_dir)

# DataFrame을 csv 파일로 저장
gender_style_counts.to_csv(csv_file_path, index=False)

# 성공 메시지 출력
print('저장 성공!!\n경로 :', csv_file_path)
```

```
저장 성공!!
경로 : ./output/1_1_valid_images.csv
```

Mission 1-2

1-2. ResNet-18를 활용하여 “성별 & 스타일” 단위로 클래스 분류를 수행하고 Validation 데이터에 대한 정확도를 제시한다.

ResNet-18 정의하기

https://github.com/pytorch/vision/blob/main/torchvision/models/resnet.py 참고

ResNet-18 Sample

In [47]:
```
# 시드 고정 함수
def set_seed(seed=42):
    torch.manual_seed(seed)
    torch.cuda.manual_seed(seed)
    torch.backends.cudnn.deterministic = True
    torch.backends.cudnn.benchmark = False
    random.seed(seed)
    np.random.seed(seed)

set_seed()  # 시드 고정
```

모든 Image dataset들을 segmentation하여 새로 저장 (pretrained DeepLab v3 사용)

In [13]:
```
# 모델 로드 (DeepLabV3)
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
pre_model = models.segmentation.deeplabv3_resnet101(weights="DEFAULT").to
pre_model.eval()

# 이미지 전처리 함수
def preprocess_image(image_path):
    input_image = Image.open(image_path).convert("RGB")
    preprocess = transforms.Compose([
        transforms.Resize((520, 520)),  # DeeplabV3의 기본 입력 크기
        transforms.ToTensor(),
        transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.22
    ])
    input_tensor = preprocess(input_image)
    input_batch = input_tensor.unsqueeze(0)  # 배치 차원 추가
    return input_batch

# Segmentation 함수
def segment_person(image_path):
    input_tensor = preprocess_image(image_path).to(device)
    with torch.no_grad():
        output = pre_model(input_tensor)['out'][0]  # 모델의 출력

    output_predictions = output.argmax(0).cpu().numpy()  # 예측된 클래스 번호

    # 사람에 해당하는 클래스(15)의 마스크 생성
    person_mask = np.where(output_predictions == 15, 1, 0).astype(np.uint

    # 여러 사람 영역 분리
    num_labels, labels_im = cv2.connectedComponents(person_mask)

    # 가장 큰 사람 영역 선택
    max_label = 1 + np.argmax([np.sum(labels_im == i) for i in range(1, n
    best_person_mask = (labels_im == max_label).astype(np.uint8)

    # 바운딩 박스 계산
    y_indices, x_indices = np.where(best_person_mask == 1)
```

```
    min_x, max_x = x_indices.min(), x_indices.max()
    min_y, max_y = y_indices.min(), y_indices.max()
    bbox = (min_x, min_y, max_x, max_y)  # 바운딩 박스 좌표

    return best_person_mask, bbox

# 사람 영역을 crop하고 520x520으로 리사이즈
def crop_and_resize_person(image_path, person_mask, bbox, target_size=(52
    original_image = Image.open(image_path).convert("RGB").resize((520, 5
    original_array = np.array(original_image)

    # 바운딩 박스 기준으로 이미지와 마스크를 crop
    min_x, min_y, max_x, max_y = bbox
    cropped_image = original_array[min_y:max_y, min_x:max_x]
    cropped_mask = person_mask[min_y:max_y, min_x:max_x]

    # 사람 영역이 전체 이미지의 1/5 미만인 경우 조건 적용
    mask_area = np.sum(person_mask)
    image_area = person_mask.shape[0] * person_mask.shape[1]
    if mask_area < image_area / 10:
        # 중앙 기준 가로 3/5, 세로 전체로 crop
        center_x = original_array.shape[1] // 2
        width_3_5 = original_array.shape[1] * 3 // 5
        min_x = max(center_x - width_3_5 // 2, 0)
        max_x = min(center_x + width_3_5 // 2, original_array.shape[1])
        cropped_image = original_array[:, min_x:max_x]

        # RGBA 포맷으로 변환 (배경 투명)
        transparent_image = np.zeros((cropped_image.shape[0], cropped_ima
        transparent_image[..., :3] = cropped_image
        transparent_image[..., 3] = 255  # 불투명하게 설정
    else:
        # mask 영역만 남기고 나머지 투명하게 처리
        transparent_image = np.zeros((cropped_image.shape[0], cropped_ima
        transparent_image[..., :3] = cropped_image
        transparent_image[..., 3] = cropped_mask * 255  # 사람 영역을 불투명

    # 520x520 크기로 리사이즈
    resized_image = Image.fromarray(transparent_image, 'RGBA').resize(tar
    return resized_image

# 디렉토리 내 모든 이미지 처리 후 저장
def process_directory(input_dir, output_dir):
    os.makedirs(output_dir, exist_ok=True)

    # tqdm을 사용하여 진행 상황을 표시
    for filename in tqdm(os.listdir(input_dir), desc="Processing Images",
        if filename.endswith('.jpg'):
            image_path = os.path.join(input_dir, filename)
            person_mask, bbox = segment_person(image_path)
            resized_image = crop_and_resize_person(image_path, person_mas

            # 알파 채널을 제거하여 RGB로 변환
            resized_image = resized_image.convert("RGB")

            # 저장 경로 설정
            output_path = os.path.join(output_dir, filename)
            resized_image.save(output_path, format='JPEG')
```

Train image 전처리

In [14]:
```
# 실행 코드
input_dir = '/home/user/Desktop/ICT/data/training_image'  # 이미지 디렉토리
output_dir = '/home/user/Desktop/ICT/data/final_seg_train_image'  # 결과 저
process_directory(input_dir, output_dir)
```

```
Processing Images:   0%|          | 0/4070 [00:00<?, ?image/s]
```

Valid image 전처리

In [15]:
```
# 실행 코드
input_dir = '/home/user/Desktop/ICT/data/validation_image'  # 이미지 디렉토리
output_dir = '/home/user/Desktop/ICT/data/final_seg_valid_image'  # 결과 저
process_directory(input_dir, output_dir)
```

```
Processing Images:   0%|          | 0/951 [00:00<?, ?image/s]
```

In [16]:
```
# 딥러닝 하이퍼파라미터를 변수로 정의 (대문자로 변경)

# 학습 파라미터
LEARNING_RATE = 0.05       # 학습률
BATCH_SIZE = 32              # 배치 크기
NUM_EPOCHS = 500             # 에포크 수
OPTIMIZER = 'AdamW'          # 최적화 알고리즘 (Adam, SGD, AdamW 등)
WEIGHT_DECAY = 1e-4          # AdamW와 같은 옵티마이저에서의 가중치 감쇠
MOMENTUM = 0.98              # SGD와 같은 옵티마이저에서의 모멘텀

# 모델 파라미터
NUM_CLASSES = 31             # 분류할 클래스 수

# 데이터셋 파라미터
TRAIN_DATA_DIR = './data/final_seg_train_image/'  # 학습 데이터 경로
VALID_DATA_DIR = './data/final_seg_valid_image'  # 검증 데이터 경로

# 장치 파라미터
DEVICE = 'cuda' if torch.cuda.is_available() else 'cpu'  # GPU 또는 CPU 사
```

분류기 생성

In [53]:
```
# 분류할 클래스 수
NUM_CLASSES = 31

# ResNet-18 모델 불러오기 (weights=None로 초기화된 모델 사용)
model = models.resnet18(weights=None)

# He 초기화를 위한 함수 정의
def init_weights_he(model):
    if isinstance(model, nn.Conv2d) or isinstance(model, nn.Linear):
        torch.nn.init.kaiming_normal_(model.weight, mode='fan_out', nonli
        if model.bias is not None:
            torch.nn.init.constant_(model.bias, 0)

# 모델의 모든 레이어에 대해 He 초기화 적용
model.apply(init_weights_he)
```

```
# 출력 클래스를 원하는 개수로 변경 (예: 31개의 클래스를 분류하는 경우)
model.fc = nn.Linear(model.fc.in_features, NUM_CLASSES)

# 마지막 fully connected 레이어에 초기화 적용
torch.nn.init.kaiming_normal_(model.fc.weight, mode='fan_out', nonlineari
torch.nn.init.constant_(model.fc.bias, 0)

# GPU 사용 가능 여부 확인
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

# 모델을 GPU로 이동
model = model.to(device)

# 모델 구조 확인
summary(model, (3, 384, 384))
```

Layer (type)	Output Shape	Param #
Conv2d-1	[-1, 64, 192, 192]	9,408
BatchNorm2d-2	[-1, 64, 192, 192]	128
ReLU-3	[-1, 64, 192, 192]	0
MaxPool2d-4	[-1, 64, 96, 96]	0
Conv2d-5	[-1, 64, 96, 96]	36,864
BatchNorm2d-6	[-1, 64, 96, 96]	128
ReLU-7	[-1, 64, 96, 96]	0
Conv2d-8	[-1, 64, 96, 96]	36,864
BatchNorm2d-9	[-1, 64, 96, 96]	128
ReLU-10	[-1, 64, 96, 96]	0
BasicBlock-11	[-1, 64, 96, 96]	0
Conv2d-12	[-1, 64, 96, 96]	36,864
BatchNorm2d-13	[-1, 64, 96, 96]	128
ReLU-14	[-1, 64, 96, 96]	0
Conv2d-15	[-1, 64, 96, 96]	36,864
BatchNorm2d-16	[-1, 64, 96, 96]	128
ReLU-17	[-1, 64, 96, 96]	0
BasicBlock-18	[-1, 64, 96, 96]	0
Conv2d-19	[-1, 128, 48, 48]	73,728
BatchNorm2d-20	[-1, 128, 48, 48]	256
ReLU-21	[-1, 128, 48, 48]	0
Conv2d-22	[-1, 128, 48, 48]	147,456
BatchNorm2d-23	[-1, 128, 48, 48]	256
Conv2d-24	[-1, 128, 48, 48]	8,192
BatchNorm2d-25	[-1, 128, 48, 48]	256
ReLU-26	[-1, 128, 48, 48]	0
BasicBlock-27	[-1, 128, 48, 48]	0
Conv2d-28	[-1, 128, 48, 48]	147,456
BatchNorm2d-29	[-1, 128, 48, 48]	256
ReLU-30	[-1, 128, 48, 48]	0
Conv2d-31	[-1, 128, 48, 48]	147,456
BatchNorm2d-32	[-1, 128, 48, 48]	256
ReLU-33	[-1, 128, 48, 48]	0
BasicBlock-34	[-1, 128, 48, 48]	0
Conv2d-35	[-1, 256, 24, 24]	294,912
BatchNorm2d-36	[-1, 256, 24, 24]	512
ReLU-37	[-1, 256, 24, 24]	0
Conv2d-38	[-1, 256, 24, 24]	589,824
BatchNorm2d-39	[-1, 256, 24, 24]	512
Conv2d-40	[-1, 256, 24, 24]	32,768
BatchNorm2d-41	[-1, 256, 24, 24]	512
ReLU-42	[-1, 256, 24, 24]	0
BasicBlock-43	[-1, 256, 24, 24]	0
Conv2d-44	[-1, 256, 24, 24]	589,824
BatchNorm2d-45	[-1, 256, 24, 24]	512
ReLU-46	[-1, 256, 24, 24]	0
Conv2d-47	[-1, 256, 24, 24]	589,824
BatchNorm2d-48	[-1, 256, 24, 24]	512
ReLU-49	[-1, 256, 24, 24]	0
BasicBlock-50	[-1, 256, 24, 24]	0
Conv2d-51	[-1, 512, 12, 12]	1,179,648
BatchNorm2d-52	[-1, 512, 12, 12]	1,024
ReLU-53	[-1, 512, 12, 12]	0
Conv2d-54	[-1, 512, 12, 12]	2,359,296
BatchNorm2d-55	[-1, 512, 12, 12]	1,024
Conv2d-56	[-1, 512, 12, 12]	131,072
BatchNorm2d-57	[-1, 512, 12, 12]	1,024

```
              ReLU-58          [-1, 512, 12, 12]               0
        BasicBlock-59          [-1, 512, 12, 12]               0
            Conv2d-60          [-1, 512, 12, 12]       2,359,296
       BatchNorm2d-61          [-1, 512, 12, 12]           1,024
              ReLU-62          [-1, 512, 12, 12]               0
            Conv2d-63          [-1, 512, 12, 12]       2,359,296
       BatchNorm2d-64          [-1, 512, 12, 12]           1,024
              ReLU-65          [-1, 512, 12, 12]               0
        BasicBlock-66          [-1, 512, 12, 12]               0
AdaptiveAvgPool2d-67            [-1, 512, 1, 1]               0
            Linear-68                  [-1, 31]          15,903
================================================================
Total params: 11,192,415
Trainable params: 11,192,415
Non-trainable params: 0
----------------------------------------------------------------
Input size (MB): 1.69
Forward/backward pass size (MB): 184.50
Params size (MB): 42.70
Estimated Total Size (MB): 228.89
----------------------------------------------------------------
```

Dataset구축하기

In [54]:
```
# Gender와 Style을 결합하여 새로운 Class 열 생성
train_df['Class'] = train_df['Gender'] + '_' + train_df['Style']
valid_df['Class'] = valid_df['Gender'] + '_' + valid_df['Style']

# 각 클래스에 고유한 숫자 인덱스 부여 (Class 열을 카테고리로 변환 후 숫자 할당)
class_mapping = {cls: idx for idx, cls in enumerate(train_df['Class'].uni
train_df['Class_Index'] = train_df['Class'].map(class_mapping)
valid_df['Class_Index'] = valid_df['Class'].map(class_mapping)

# File_Name과 인덱스를 결합
train_dataset = train_df[['File_Name', 'Class_Index']]
valid_dataset = valid_df[['File_Name', 'Class_Index']]

# 결과 CSV 파일로 저장
train_csv_file = '1train_dataset.csv'
valid_csv_file = '1valid_dataset.csv'

train_dataset.to_csv(train_csv_file, index=False)
valid_dataset.to_csv(valid_csv_file, index=False)
```

In [55]:
```
class_mapping
```

Out[55]:
```
{'여성_bodyconscious': 0,
 '남성_normcore': 1,
 '여성_disco': 2,
 '여성_sportivecasual': 3,
 '여성_feminine': 4,
 '남성_bold': 5,
 '여성_cityglam': 6,
 '남성_hippie': 7,
 '여성_genderless': 8,
 '여성_kitsch': 9,
 '여성_powersuit': 10,
 '남성_hiphop': 11,
 '남성_mods': 12,
 '남성_ivy': 13,
 '여성_ecology': 14,
 '여성_athleisure': 15,
 '남성_sportivecasual': 16,
 '여성_hiphop': 17,
 '여성_hippie': 18,
 '여성_normcore': 19,
 '여성_oriental': 20,
 '남성_metrosexual': 21,
 '여성_classic': 22,
 '여성_punk': 23,
 '여성_lingerie': 24,
 '여성_grunge': 25,
 '여성_minimal': 26,
 '여성_lounge': 27,
 '여성_space': 28,
 '여성_popart': 29,
 '여성_military': 30}
```

In [56]:
```
# indexing으로 실제 class name을 찾을 수 있는 list
class_names = list(class_mapping.keys())
```

Final_seg_train_image의 Mean, Std 계산하기

In [57]:
```
# 이미지 디렉토리 경로 설정
img_dir = './data/final_seg_train_image/'

# 이미지 전처리 (384*384로 리사이즈하고, Normalize 없이 텐서로 변환)
transform = transforms.Compose([
    transforms.Resize((384, 384)),
    transforms.ToTensor()
])

# RGB 채널별 합을 계산하기 위한 변수 초기화
mean = torch.zeros(3)
std = torch.zeros(3)
num_samples = 0

# 디렉토리 순회하며 이미지 로드
for img_file in os.listdir(img_dir):
    if img_file.lower().endswith(('.png', '.jpg', '.jpeg')):
        img_path = os.path.join(img_dir, img_file)

        # 이미지 로드 및 전처리
        image = Image.open(img_path).convert("RGB")
        image = transform(image)  # [C, H, W] 형식의 텐서로 변환
```

```
        # 전체 픽셀 수 업데이트
        num_samples += image.size(1) * image.size(2)  # Height x Width

        # 채널별 합 및 표준 편차 계산
        mean += image.mean([1, 2])  # [1, 2]는 Height와 Width 차원
        std += image.std([1, 2])    # [1, 2]는 Height와 Width 차원

# 평균 및 표준 편차 계산
mean /= len(os.listdir(img_dir))
std /= len(os.listdir(img_dir))

print(f"Mean (RGB) for train images: {mean}")
print(f"Std (RGB) for train images: {std}")
```

transform에 적용하세요

In [58]:
```
# CustomImageDataset 클래스 정의
class CustomImageDataset(Dataset):
    def __init__(self, csv_file, img_dir, transform=None):
        self.data_frame = pd.read_csv(csv_file)
        self.img_dir = img_dir
        self.transform = transform

    def __len__(self):
        return len(self.data_frame)

    def __getitem__(self, idx):
        img_name = os.path.join(self.img_dir, self.data_frame.iloc[idx, 0
        image = Image.open(img_name).convert("RGB")
        if self.transform:
            image = self.transform(image)
        label = int(self.data_frame.iloc[idx, 1])
        return image, label
```

In [59]:
```
# 각 전처리 및 증강 기법 정의 384
train_transform = transforms.Compose([
    transforms.Resize((384, 384)),
    transforms.ToTensor(),
    transforms.Normalize([0.2651, 0.2375, 0.2316], [0.2848, 0.2609, 0.255
])

train_HorizontalFlip_transform = transforms.Compose([
    transforms.Resize((384, 384)),
    transforms.RandomHorizontalFlip(p=0.8),  # 80% 확률로 수평 반전
    transforms.RandomApply([transforms.RandomRotation(degrees=15)], p=0.2
    transforms.ColorJitter(
        brightness=0.2,  # 밝기 조정 (0.2 범위 내에서 밝기를 랜덤하게 조절)
        hue=0.1          # 색조 조정 (0.1 범위 내에서 색조를 랜덤하게 조절)
    ),
    transforms.ToTensor(),
    transforms.Normalize([0.2651, 0.2375, 0.2316], [0.2848, 0.2609, 0.255
])

val_transform = transforms.Compose([
    transforms.Resize((384, 384)),
```

```
    transforms.ToTensor(),
    transforms.Normalize([0.2651, 0.2375, 0.2316], [0.2848, 0.2609, 0.255
])

# 각 데이터셋 로드
train_dataset = CustomImageDataset(csv_file=train_csv_file, img_dir=TRAIN
train_HorizontalFlip_dataset = CustomImageDataset (csv_file=train_csv_fil

val_dataset = CustomImageDataset(csv_file=valid_csv_file, img_dir=VALID_D

# 여러 데이터셋을 하나로 병합
concat_dataset = ConcatDataset([train_dataset, train_HorizontalFlip_datas

# DataLoader 생성
train_loader = DataLoader(concat_dataset, batch_size=16, shuffle=True, nu
valid_loader = DataLoader(val_dataset, batch_size=32, shuffle=False, num_
```

In [60]:
```
# 손실 함수 및 옵티마이저 설정
criterion = nn.CrossEntropyLoss()  # 분류 작업을 위한 손실 함수 (One-Hot-Encod
optimizer = optim.AdamW(model.parameters(), lr=LEARNING_RATE, weight_deca

# ReduceLROnPlateau 스케줄러 설정 (검증 손실이 2번 개선되지 않으면 학습률 감소)
scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='min', p
```

In [61]:
```
# Accuracy 기반의 early stopping class
class EarlyStopping:
    def __init__(self, patience=5, min_delta=0):
        self.patience = patience
        self.min_delta = min_delta
        self.counter = 0
        self.best_accuracy = None
        self.early_stop = False

    def __call__(self, val_accuracy):
        if self.best_accuracy is None:
            self.best_accuracy = val_accuracy
        elif val_accuracy < self.best_accuracy + self.min_delta:  # 정확도
            self.counter += 1
            if self.counter >= self.patience:
                self.early_stop = True
        else:
            self.best_accuracy = val_accuracy
            self.counter = 0


# Early Stopping 인스턴스 생성
early_stopping = EarlyStopping(patience=20, min_delta=0.1) # accuracy 기반
```

In [62]:
```
def train_one_epoch(model, train_loader, criterion, optimizer, scheduler,
    model.train()
    running_loss = 0.0
    correct = 0
    total = 0

    progress_bar = tqdm(train_loader, desc='Training', leave=False, total

    for images, labels in progress_bar:
        images, labels = images.to(device), labels.to(device)
```

```
        optimizer.zero_grad()

        # 모델 예측
        outputs = model(images)

        # 손실 계산
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        # 손실 누적
        running_loss += loss.item() * images.size(0)

        # 예측값과 실제값으로 정확도 계산 (argmax 사용)
        _, predicted = torch.max(outputs, 1)
        correct += (predicted == labels).sum().item()
        total += labels.size(0)

        # tqdm 진행 상황 업데이트
        progress_bar.set_postfix(loss=running_loss / total, accuracy=100

    # 에폭이 끝난 후 한 번만 최종 손실 및 정확도 출력
    epoch_loss = running_loss / total
    epoch_accuracy = 100 * correct / total

    # 최종 결과 로그 출력
    #tqdm.write(f'Final Epoch Loss: {epoch_loss:.4f}, Final Accuracy: {ep

    #scheduler.step()

    return epoch_loss, epoch_accuracy
```

In [63]:

```
def validate(model, valid_loader, criterion, device):
    model.eval()
    running_loss = 0.0
    correct = 0
    total = 0

    # tqdm 진행 바 설정
    progress_bar = tqdm(valid_loader, desc='Validation', leave=False)

    with torch.no_grad():
        for images, labels in progress_bar:
            images, labels = images.to(device), labels.to(device)

            # 모델 예측 (logits 사용)
            outputs = model(images)

            # 손실 계산
            loss = criterion(outputs, labels)

            # 손실 누적
            running_loss += loss.item() * images.size(0)

            # 예측값과 실제값으로 정확도 계산 (argmax 사용)
            _, predicted = torch.max(outputs, 1)
            correct += (predicted == labels).sum().item()
            total += labels.size(0)

            # tqdm 진행 상황 업데이트
```

```
            progress_bar.set_postfix(loss=running_loss / total, accuracy=

    # 에폭이 끝난 후 한 번만 최종 손실 및 정확도 출력
    val_loss = running_loss / total
    val_accuracy = 100 * correct / total

    # 최종 결과 로그 출력
    #tqdm.write(f'Final Validation Loss: {val_loss:.4f}, Final Accuracy:

    return val_loss, val_accuracy
```

In [64]:

```
# Output 디렉토리 생성 (존재하지 않으면 생성)
output_dir = './output'
if not os.path.exists(output_dir):
    os.makedirs(output_dir)

# 학습 및 검증 과정을 기록하기 위한 리스트
train_losses = []
train_accuracies = []
val_losses = []
val_accuracies = []

# 최적의 val_loss 초기값 설정
best_val_loss = float('inf')
best_accuracy = 0

# tqdm를 이용한 에포크 진행 표시
epoch_bar = tqdm(range(NUM_EPOCHS), desc="Total Epoch Progress", total=NU

# 전체 학습 루프
for epoch in epoch_bar:
    print(f'Epoch {epoch + 1}/{NUM_EPOCHS}')

    # 한 epoch 동안의 학습 진행
    train_loss, train_acc = train_one_epoch(model, train_loader, criterio
    train_losses.append(train_loss)
    train_accuracies.append(train_acc)

    # 검증 단계 진행
    val_loss, val_acc = validate(model, valid_loader, criterion, device)
    val_losses.append(val_loss)
    val_accuracies.append(val_acc)

    # 결과 출력
    print(f'Epoch [{epoch+1}/{NUM_EPOCHS}] - Train Loss: {train_loss:.4f}

    # 최적의 val_accuracy를 기록하고 모델 저장 (추가 정보 포함)
    if val_acc > best_accuracy:
        best_accuracy = val_acc
        best_model_path = os.path.join(output_dir, 'best_model.pth')
        torch.save({
            'epoch': epoch + 1,
            'model_state_dict': model.state_dict(),
            'best_accuracy': best_accuracy,
            'optimizer_state_dict': optimizer.state_dict(),
            'scheduler_state_dict': scheduler.state_dict()
        }, best_model_path)
        print(f'Best model saved to {best_model_path} with val_accuracy:
```

```
    # Early Stopping 체크
    # early_stopping(val_loss)
    early_stopping(val_acc)
    if early_stopping.early_stop:
        print("Early stopping triggered!")
        break  # 학습 중지

    # 검증 손실을 기반으로 스케줄러 업데이트
    scheduler.step(val_loss)

# 학습이 끝난 후 최종 모델 저장
final_model_path = os.path.join(output_dir, 'final_model.pth')
torch.save({
    'epoch': epoch + 1,
    'model_state_dict': model.state_dict(),
    'optimizer_state_dict': optimizer.state_dict(),
    'scheduler_state_dict': scheduler.state_dict()
}, final_model_path)
print(f'Final model saved to {final_model_path}')
```

```
Total Epoch Progress:   0%|          | 0/500 [00:00<?, ?it/s]
Epoch 1/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [1/500] - Train Loss: 3.7479, Train Acc: 7.91% | Val Loss: 3.1733, V
al Acc: 7.57%
Best model saved to ./output/best_model.pth with val_accuracy: 7.57%
Epoch 2/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [2/500] - Train Loss: 3.1712, Train Acc: 8.45% | Val Loss: 3.1922, V
al Acc: 7.26%
Epoch 3/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [3/500] - Train Loss: 3.1619, Train Acc: 8.28% | Val Loss: 3.1592, V
al Acc: 8.31%
Best model saved to ./output/best_model.pth with val_accuracy: 8.31%
Epoch 4/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [4/500] - Train Loss: 3.1518, Train Acc: 8.61% | Val Loss: 3.1344, V
al Acc: 9.78%
Best model saved to ./output/best_model.pth with val_accuracy: 9.78%
Epoch 5/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [5/500] - Train Loss: 3.1394, Train Acc: 8.66% | Val Loss: 3.1541, V
al Acc: 8.20%
Epoch 6/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [6/500] - Train Loss: 3.1274, Train Acc: 9.23% | Val Loss: 3.1287, V
al Acc: 8.73%
Epoch 7/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [7/500] - Train Loss: 3.1121, Train Acc: 9.03% | Val Loss: 3.1075, V
al Acc: 8.31%
Epoch 8/500
```

```
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [8/500] - Train Loss: 3.0936, Train Acc: 9.90% | Val Loss: 3.1122, V
al Acc: 10.20%
Best model saved to ./output/best_model.pth with val_accuracy: 10.20%
Epoch 9/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [9/500] - Train Loss: 3.0828, Train Acc: 10.23% | Val Loss: 3.0848,
Val Acc: 7.26%
Epoch 10/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [10/500] - Train Loss: 3.0598, Train Acc: 10.95% | Val Loss: 3.1506,
Val Acc: 8.83%
Epoch 11/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [11/500] - Train Loss: 3.0542, Train Acc: 10.31% | Val Loss: 3.1006,
Val Acc: 10.41%
Best model saved to ./output/best_model.pth with val_accuracy: 10.41%
Epoch 12/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [12/500] - Train Loss: 3.0307, Train Acc: 11.09% | Val Loss: 3.0758,
Val Acc: 10.30%
Epoch 13/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [13/500] - Train Loss: 3.0287, Train Acc: 10.84% | Val Loss: 3.0160,
Val Acc: 10.73%
Best model saved to ./output/best_model.pth with val_accuracy: 10.73%
Epoch 14/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [14/500] - Train Loss: 3.0012, Train Acc: 11.27% | Val Loss: 3.0877,
Val Acc: 8.10%
Epoch 15/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [15/500] - Train Loss: 2.9720, Train Acc: 11.92% | Val Loss: 3.0345,
Val Acc: 10.41%
Epoch 16/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [16/500] - Train Loss: 2.9379, Train Acc: 12.40% | Val Loss: 2.9456,
Val Acc: 12.93%
Best model saved to ./output/best_model.pth with val_accuracy: 12.93%
Epoch 17/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [17/500] - Train Loss: 2.8967, Train Acc: 12.97% | Val Loss: 3.5314,
Val Acc: 8.41%
Epoch 18/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [18/500] - Train Loss: 2.8663, Train Acc: 13.54% | Val Loss: 2.9378,
Val Acc: 15.25%
Best model saved to ./output/best_model.pth with val_accuracy: 15.25%
Epoch 19/500
```

```
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [19/500] - Train Loss: 2.8216, Train Acc: 14.62% | Val Loss: 2.8611,
Val Acc: 14.30%
Epoch 20/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [20/500] - Train Loss: 2.7744, Train Acc: 14.66% | Val Loss: 2.7245,
Val Acc: 17.98%
Best model saved to ./output/best_model.pth with val_accuracy: 17.98%
Epoch 21/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [21/500] - Train Loss: 2.7329, Train Acc: 15.33% | Val Loss: 2.7366,
Val Acc: 15.67%
Epoch 22/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [22/500] - Train Loss: 2.6857, Train Acc: 16.27% | Val Loss: 2.6949,
Val Acc: 17.67%
Epoch 23/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [23/500] - Train Loss: 2.6414, Train Acc: 17.57% | Val Loss: 2.6211,
Val Acc: 17.67%
Epoch 24/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [24/500] - Train Loss: 2.5786, Train Acc: 19.46% | Val Loss: 2.6229,
Val Acc: 17.77%
Epoch 25/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [25/500] - Train Loss: 2.5303, Train Acc: 20.28% | Val Loss: 2.7831,
Val Acc: 16.82%
Epoch 26/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [26/500] - Train Loss: 2.4603, Train Acc: 21.45% | Val Loss: 2.5614,
Val Acc: 21.56%
Best model saved to ./output/best_model.pth with val_accuracy: 21.56%
Epoch 27/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [27/500] - Train Loss: 2.3900, Train Acc: 23.83% | Val Loss: 2.6460,
Val Acc: 21.14%
Epoch 28/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [28/500] - Train Loss: 2.3124, Train Acc: 25.39% | Val Loss: 2.5627,
Val Acc: 23.45%
Best model saved to ./output/best_model.pth with val_accuracy: 23.45%
Epoch 29/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [29/500] - Train Loss: 2.2286, Train Acc: 27.58% | Val Loss: 2.8310,
Val Acc: 18.61%
Epoch 30/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
```

```
Epoch [30/500] - Train Loss: 2.1225, Train Acc: 31.27% | Val Loss: 2.4400,
Val Acc: 30.18%
Best model saved to ./output/best_model.pth with val_accuracy: 30.18%
Epoch 31/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [31/500] - Train Loss: 1.9988, Train Acc: 34.94% | Val Loss: 2.3536,
Val Acc: 31.23%
Best model saved to ./output/best_model.pth with val_accuracy: 31.23%
Epoch 32/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [32/500] - Train Loss: 1.8860, Train Acc: 38.62% | Val Loss: 2.3594,
Val Acc: 32.60%
Best model saved to ./output/best_model.pth with val_accuracy: 32.60%
Epoch 33/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [33/500] - Train Loss: 1.7880, Train Acc: 41.65% | Val Loss: 2.2399,
Val Acc: 39.54%
Best model saved to ./output/best_model.pth with val_accuracy: 39.54%
Epoch 34/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [34/500] - Train Loss: 1.6797, Train Acc: 45.47% | Val Loss: 2.2421,
Val Acc: 42.48%
Best model saved to ./output/best_model.pth with val_accuracy: 42.48%
Epoch 35/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [35/500] - Train Loss: 1.5552, Train Acc: 50.05% | Val Loss: 2.5003,
Val Acc: 36.38%
Epoch 36/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [36/500] - Train Loss: 1.4343, Train Acc: 54.83% | Val Loss: 2.3652,
Val Acc: 43.74%
Best model saved to ./output/best_model.pth with val_accuracy: 43.74%
Epoch 37/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [37/500] - Train Loss: 1.3300, Train Acc: 58.40% | Val Loss: 3.5640,
Val Acc: 28.18%
Epoch 38/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [38/500] - Train Loss: 1.2866, Train Acc: 60.17% | Val Loss: 2.1659,
Val Acc: 52.58%
Best model saved to ./output/best_model.pth with val_accuracy: 52.58%
Epoch 39/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [39/500] - Train Loss: 1.1959, Train Acc: 63.81% | Val Loss: 2.4693,
Val Acc: 51.42%
Epoch 40/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
```

```
Epoch [40/500] - Train Loss: 1.1204, Train Acc: 66.11% | Val Loss: 2.3912,
Val Acc: 54.15%
Best model saved to ./output/best_model.pth with val_accuracy: 54.15%
Epoch 41/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [41/500] - Train Loss: 1.0350, Train Acc: 69.73% | Val Loss: 2.3318,
Val Acc: 54.78%
Best model saved to ./output/best_model.pth with val_accuracy: 54.78%
Epoch 42/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [42/500] - Train Loss: 0.9820, Train Acc: 71.83% | Val Loss: 2.2404,
Val Acc: 56.78%
Best model saved to ./output/best_model.pth with val_accuracy: 56.78%
Epoch 43/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [43/500] - Train Loss: 0.9485, Train Acc: 73.30% | Val Loss: 2.3279,
Val Acc: 57.10%
Best model saved to ./output/best_model.pth with val_accuracy: 57.10%
Epoch 00043: reducing learning rate of group 0 to 4.5000e-02.
Epoch 44/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [44/500] - Train Loss: 0.8726, Train Acc: 75.98% | Val Loss: 2.2438,
Val Acc: 59.62%
Best model saved to ./output/best_model.pth with val_accuracy: 59.62%
Epoch 45/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [45/500] - Train Loss: 0.8109, Train Acc: 78.40% | Val Loss: 2.3496,
Val Acc: 58.15%
Epoch 46/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [46/500] - Train Loss: 0.7577, Train Acc: 79.74% | Val Loss: 2.5772,
Val Acc: 57.31%
Epoch 47/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [47/500] - Train Loss: 0.7572, Train Acc: 80.01% | Val Loss: 2.5349,
Val Acc: 54.99%
Epoch 48/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [48/500] - Train Loss: 0.7850, Train Acc: 79.34% | Val Loss: 2.3701,
Val Acc: 57.20%
Epoch 00048: reducing learning rate of group 0 to 4.0500e-02.
Epoch 49/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [49/500] - Train Loss: 0.6889, Train Acc: 82.49% | Val Loss: 2.3207,
Val Acc: 60.46%
Best model saved to ./output/best_model.pth with val_accuracy: 60.46%
Epoch 50/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
```

```
Epoch [50/500] - Train Loss: 0.6609, Train Acc: 84.39% | Val Loss: 2.2970,
Val Acc: 61.41%
Best model saved to ./output/best_model.pth with val_accuracy: 61.41%
Epoch 51/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [51/500] - Train Loss: 0.6665, Train Acc: 83.61% | Val Loss: 2.4207,
Val Acc: 61.62%
Best model saved to ./output/best_model.pth with val_accuracy: 61.62%
Epoch 52/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [52/500] - Train Loss: 0.6178, Train Acc: 85.52% | Val Loss: 2.4396,
Val Acc: 62.15%
Best model saved to ./output/best_model.pth with val_accuracy: 62.15%
Epoch 53/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [53/500] - Train Loss: 0.5804, Train Acc: 85.52% | Val Loss: 2.5711,
Val Acc: 60.36%
Epoch 00053: reducing learning rate of group 0 to 3.6450e-02.
Epoch 54/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [54/500] - Train Loss: 0.5825, Train Acc: 86.81% | Val Loss: 2.4185,
Val Acc: 62.04%
Epoch 55/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [55/500] - Train Loss: 0.5573, Train Acc: 87.65% | Val Loss: 2.3968,
Val Acc: 61.51%
Epoch 56/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [56/500] - Train Loss: 0.5390, Train Acc: 88.24% | Val Loss: 2.4591,
Val Acc: 61.30%
Epoch 57/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [57/500] - Train Loss: 0.5317, Train Acc: 87.85% | Val Loss: 2.4266,
Val Acc: 61.30%
Epoch 58/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [58/500] - Train Loss: 0.5254, Train Acc: 87.90% | Val Loss: 2.5672,
Val Acc: 60.99%
Epoch 00058: reducing learning rate of group 0 to 3.2805e-02.
Epoch 59/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [59/500] - Train Loss: 0.4608, Train Acc: 90.14% | Val Loss: 2.5069,
Val Acc: 61.51%
Epoch 60/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [60/500] - Train Loss: 0.4999, Train Acc: 89.31% | Val Loss: 2.5689,
Val Acc: 61.30%
Epoch 61/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
```

```
Epoch [61/500] - Train Loss: 0.5043, Train Acc: 89.14% | Val Loss: 2.5910,
Val Acc: 59.94%
Epoch 62/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [62/500] - Train Loss: 0.5189, Train Acc: 88.86% | Val Loss: 2.5436,
Val Acc: 62.36%
Best model saved to ./output/best_model.pth with val_accuracy: 62.36%
Epoch 63/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [63/500] - Train Loss: 0.5285, Train Acc: 88.72% | Val Loss: 2.2885,
Val Acc: 63.20%
Best model saved to ./output/best_model.pth with val_accuracy: 63.20%
Epoch 00063: reducing learning rate of group 0 to 2.9525e-02.
Epoch 64/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [64/500] - Train Loss: 0.4921, Train Acc: 89.73% | Val Loss: 2.3686,
Val Acc: 62.99%
Epoch 65/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [65/500] - Train Loss: 0.4263, Train Acc: 90.95% | Val Loss: 2.3970,
Val Acc: 62.25%
Epoch 66/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [66/500] - Train Loss: 0.4351, Train Acc: 90.71% | Val Loss: 2.3776,
Val Acc: 62.46%
Epoch 67/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [67/500] - Train Loss: 0.5097, Train Acc: 88.97% | Val Loss: 2.4015,
Val Acc: 61.62%
Epoch 68/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [68/500] - Train Loss: 0.4519, Train Acc: 90.58% | Val Loss: 2.4010,
Val Acc: 62.99%
Epoch 00068: reducing learning rate of group 0 to 2.6572e-02.
Epoch 69/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [69/500] - Train Loss: 0.4090, Train Acc: 91.52% | Val Loss: 2.5774,
Val Acc: 61.62%
Epoch 70/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [70/500] - Train Loss: 0.4022, Train Acc: 91.17% | Val Loss: 2.4275,
Val Acc: 62.46%
Epoch 71/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [71/500] - Train Loss: 0.4405, Train Acc: 91.08% | Val Loss: 2.3068,
Val Acc: 62.67%
Epoch 72/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
```

```
Epoch [72/500] - Train Loss: 0.3982, Train Acc: 91.93% | Val Loss: 2.5062,
Val Acc: 62.57%
Epoch 73/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [73/500] - Train Loss: 0.4386, Train Acc: 90.75% | Val Loss: 2.3681,
Val Acc: 63.72%
Best model saved to ./output/best_model.pth with val_accuracy: 63.72%
Epoch 00073: reducing learning rate of group 0 to 2.3915e-02.
Epoch 74/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [74/500] - Train Loss: 0.3688, Train Acc: 92.17% | Val Loss: 2.4460,
Val Acc: 62.67%
Epoch 75/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [75/500] - Train Loss: 0.3833, Train Acc: 92.26% | Val Loss: 2.4530,
Val Acc: 61.93%
Epoch 76/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [76/500] - Train Loss: 0.4014, Train Acc: 91.62% | Val Loss: 2.4544,
Val Acc: 61.93%
Epoch 77/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [77/500] - Train Loss: 0.3962, Train Acc: 91.54% | Val Loss: 2.3257,
Val Acc: 62.15%
Epoch 78/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [78/500] - Train Loss: 0.3695, Train Acc: 92.25% | Val Loss: 2.3989,
Val Acc: 63.09%
Epoch 00078: reducing learning rate of group 0 to 2.1523e-02.
Epoch 79/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [79/500] - Train Loss: 0.3649, Train Acc: 93.06% | Val Loss: 2.3494,
Val Acc: 63.72%
Epoch 80/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [80/500] - Train Loss: 0.3721, Train Acc: 92.69% | Val Loss: 2.4949,
Val Acc: 62.99%
Epoch 81/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [81/500] - Train Loss: 0.3470, Train Acc: 92.62% | Val Loss: 2.4249,
Val Acc: 62.78%
Epoch 82/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [82/500] - Train Loss: 0.3607, Train Acc: 92.69% | Val Loss: 2.4642,
Val Acc: 63.09%
Epoch 83/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
```

```
Epoch [83/500] - Train Loss: 0.3547, Train Acc: 92.53% | Val Loss: 2.3273,
Val Acc: 63.93%
Best model saved to ./output/best_model.pth with val_accuracy: 63.93%
Epoch 00083: reducing learning rate of group 0 to 1.9371e-02.
Epoch 84/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [84/500] - Train Loss: 0.3592, Train Acc: 92.32% | Val Loss: 2.3223,
Val Acc: 63.30%
Epoch 85/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [85/500] - Train Loss: 0.3414, Train Acc: 93.03% | Val Loss: 2.5042,
Val Acc: 61.51%
Epoch 86/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [86/500] - Train Loss: 0.3229, Train Acc: 93.01% | Val Loss: 2.3128,
Val Acc: 62.57%
Epoch 87/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [87/500] - Train Loss: 0.3307, Train Acc: 93.07% | Val Loss: 2.3624,
Val Acc: 63.09%
Epoch 88/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [88/500] - Train Loss: 0.3561, Train Acc: 92.51% | Val Loss: 2.5112,
Val Acc: 62.57%
Epoch 00088: reducing learning rate of group 0 to 1.7434e-02.
Epoch 89/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [89/500] - Train Loss: 0.3390, Train Acc: 93.18% | Val Loss: 2.3488,
Val Acc: 62.36%
Epoch 90/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [90/500] - Train Loss: 0.3222, Train Acc: 93.18% | Val Loss: 2.3712,
Val Acc: 63.30%
Epoch 91/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [91/500] - Train Loss: 0.3176, Train Acc: 93.44% | Val Loss: 2.3474,
Val Acc: 63.09%
Epoch 92/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [92/500] - Train Loss: 0.3237, Train Acc: 93.16% | Val Loss: 2.3849,
Val Acc: 63.09%
Epoch 93/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [93/500] - Train Loss: 0.3388, Train Acc: 93.07% | Val Loss: 2.2879,
Val Acc: 62.78%
Epoch 00093: reducing learning rate of group 0 to 1.5691e-02.
Epoch 94/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
```

```
Epoch [94/500] - Train Loss: 0.3227, Train Acc: 93.19% | Val Loss: 2.3156,
Val Acc: 63.41%
Epoch 95/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [95/500] - Train Loss: 0.3146, Train Acc: 93.13% | Val Loss: 2.2610,
Val Acc: 63.41%
Epoch 96/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [96/500] - Train Loss: 0.3014, Train Acc: 93.64% | Val Loss: 2.3646,
Val Acc: 64.67%
Best model saved to ./output/best_model.pth with val_accuracy: 64.67%
Epoch 97/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [97/500] - Train Loss: 0.3041, Train Acc: 93.49% | Val Loss: 2.2454,
Val Acc: 63.83%
Epoch 98/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [98/500] - Train Loss: 0.2840, Train Acc: 93.85% | Val Loss: 2.2251,
Val Acc: 63.62%
Epoch 00098: reducing learning rate of group 0 to 1.4121e-02.
Epoch 99/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [99/500] - Train Loss: 0.3033, Train Acc: 93.34% | Val Loss: 2.2353,
Val Acc: 63.72%
Epoch 100/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [100/500] - Train Loss: 0.3388, Train Acc: 92.89% | Val Loss: 2.246
2, Val Acc: 62.88%
Epoch 101/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [101/500] - Train Loss: 0.2951, Train Acc: 93.37% | Val Loss: 2.267
8, Val Acc: 62.78%
Epoch 102/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [102/500] - Train Loss: 0.2909, Train Acc: 93.77% | Val Loss: 2.304
0, Val Acc: 64.04%
Epoch 103/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [103/500] - Train Loss: 0.3061, Train Acc: 93.50% | Val Loss: 2.168
6, Val Acc: 63.41%
Epoch 00103: reducing learning rate of group 0 to 1.2709e-02.
Epoch 104/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [104/500] - Train Loss: 0.2949, Train Acc: 93.43% | Val Loss: 2.232
9, Val Acc: 63.83%
Epoch 105/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
```

```
Epoch [105/500] - Train Loss: 0.2742, Train Acc: 93.88% | Val Loss: 2.315
6, Val Acc: 62.88%
Epoch 106/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [106/500] - Train Loss: 0.2690, Train Acc: 94.09% | Val Loss: 2.222
2, Val Acc: 63.30%
Epoch 107/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [107/500] - Train Loss: 0.2852, Train Acc: 93.88% | Val Loss: 2.273
7, Val Acc: 62.67%
Epoch 108/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [108/500] - Train Loss: 0.2663, Train Acc: 93.97% | Val Loss: 2.253
5, Val Acc: 62.88%
Epoch 00108: reducing learning rate of group 0 to 1.1438e-02.
Epoch 109/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [109/500] - Train Loss: 0.2884, Train Acc: 93.46% | Val Loss: 2.199
9, Val Acc: 63.09%
Epoch 110/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [110/500] - Train Loss: 0.2657, Train Acc: 94.10% | Val Loss: 2.272
2, Val Acc: 63.72%
Epoch 111/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [111/500] - Train Loss: 0.2858, Train Acc: 93.75% | Val Loss: 2.296
1, Val Acc: 63.72%
Epoch 112/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [112/500] - Train Loss: 0.2686, Train Acc: 94.13% | Val Loss: 2.238
2, Val Acc: 64.25%
Epoch 113/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [113/500] - Train Loss: 0.2781, Train Acc: 93.71% | Val Loss: 2.169
2, Val Acc: 62.67%
Epoch 00113: reducing learning rate of group 0 to 1.0295e-02.
Epoch 114/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [114/500] - Train Loss: 0.2596, Train Acc: 94.04% | Val Loss: 2.208
3, Val Acc: 63.93%
Epoch 115/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Epoch [115/500] - Train Loss: 0.2416, Train Acc: 94.25% | Val Loss: 2.165
2, Val Acc: 62.88%
Epoch 116/500
Training:   0%|          | 0/509 [00:00<?, ?it/s]
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
```

```
Epoch [116/500] - Train Loss: 0.2473, Train Acc: 94.18% | Val Loss: 2.267
0, Val Acc: 62.78%
Early stopping triggered!
Final model saved to ./output/final_model.pth
```

In [65]:
```python
# Loss 및 Accuracy 그래프 출력 및 저장
def plot_and_save_metrics(train_losses, train_accuracies, val_losses, val
    epochs = range(1, len(train_losses) + 1)

    # Loss 그래프
    plt.figure(figsize=(14, 5))

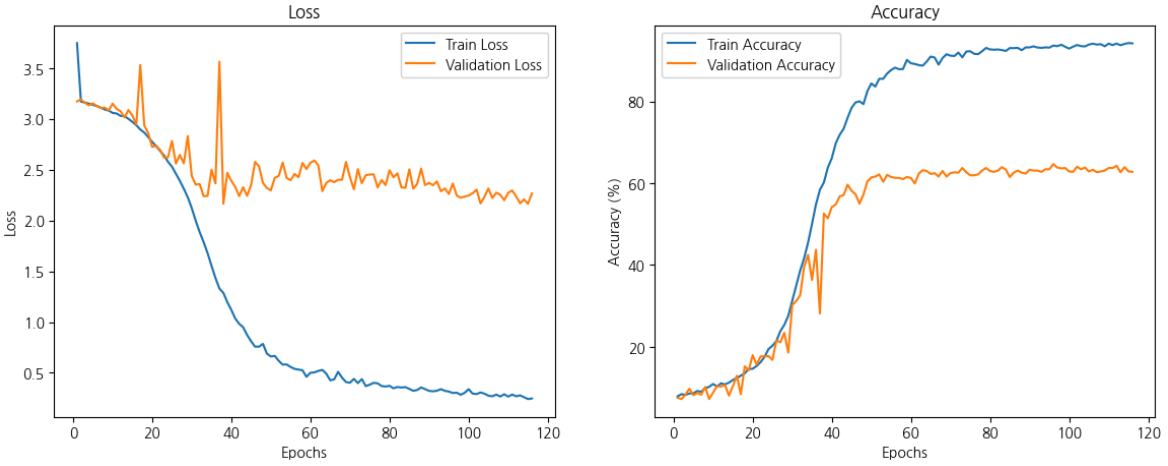
    # Loss
    plt.subplot(1, 2, 1)
    plt.plot(epochs, train_losses, label='Train Loss')
    plt.plot(epochs, val_losses, label='Validation Loss')
    plt.title('Loss')
    plt.xlabel('Epochs')
    plt.ylabel('Loss')
    plt.legend()

    # Accuracy 그래프
    plt.subplot(1, 2, 2)
    plt.plot(epochs, train_accuracies, label='Train Accuracy')
    plt.plot(epochs, val_accuracies, label='Validation Accuracy')
    plt.title('Accuracy')
    plt.xlabel('Epochs')
    plt.ylabel('Accuracy (%)')
    plt.legend()

    # 그래프 파일 저장
    plot_save_path = os.path.join(output_dir, 'training_plot.png')
    plt.savefig(plot_save_path)
    print(f'Plot saved to {plot_save_path}')
    plt.show()

# Loss 및 Accuracy 그래프 출력 및 저장
plot_and_save_metrics(train_losses, train_accuracies, val_losses, val_acc
```

```
Plot saved to ./output/training_plot.png
```



In [66]:
```python
# 시드 고정
set_seed()

# 모델 준비 및 가중치 로드
NUM_CLASSES = 31
```

```
model = models.resnet18(weights=None)

model.fc = nn.Linear(model.fc.in_features, NUM_CLASSES)


# best_model.pth에서 모든 상태를 로드
checkpoint = torch.load("/home/user/Desktop/ICT/output/best_model.pth", m
print('Saved Best Acc :',checkpoint['best_accuracy'])
model.load_state_dict(checkpoint['model_state_dict'])


model = model.to(device)
model.eval()  # 평가 모드 설정

# Validation 수행 및 결과 출력
val_loss, val_accuracy = validate(model, valid_loader, criterion, device)
print(f"Validation Loss: {val_loss:.4f}, Validation Accuracy: {val_accura
```

```
Saved Best Acc : 64.66876971608833
Validation:   0%|          | 0/30 [00:00<?, ?it/s]
Validation Loss: 2.3646, Validation Accuracy: 64.67%
```

Option. get good/bad answer in plot

In [75]:
```
# Custom Dataset 클래스 정의
class CustomImageDataset(Dataset):
    def __init__(self, csv_file, img_dir, transform=None):
        self.data_frame = pd.read_csv(csv_file)
        self.img_dir = img_dir
        self.transform = transform

    def __len__(self):
        return len(self.data_frame)

    def __getitem__(self, idx):
        img_name = self.data_frame.iloc[idx, 0]
        img_path = os.path.join(self.img_dir, img_name)  # VALID_DATA_DIR
        image = Image.open(img_path).convert("RGB")

        if self.transform:
            image = self.transform(image)

        label = int(self.data_frame.iloc[idx, 1])
        return image, label, img_path  # img_path를 반환


val_dataset = CustomImageDataset(csv_file=valid_csv_file, img_dir=VALID_D
valid_loader = DataLoader(val_dataset, batch_size=1, shuffle=False, num_w

# Validation 함수 정의
def validate_with_examples(model, valid_loader, criterion, device):
    model.eval()
    running_loss = 0.0
    correct = 0
    total = 0

    correct_examples = []
    incorrect_examples = []

    with torch.no_grad():
```

```
        for images, labels, img_paths in valid_loader:
            images, labels = images.to(device), labels.to(device)

            # 모델 예측
            outputs = model(images)
            _, predicted = torch.max(outputs, 1)

            # 손실 계산
            loss = criterion(outputs, labels)
            running_loss += loss.item()

            # 예측 결과 저장
            total += 1
            if predicted == labels:
                correct += 1
                if len(correct_examples) < 4:
                    correct_examples.append((images.cpu(), predicted.item
            else:
                if len(incorrect_examples) < 4:
                    incorrect_examples.append((images.cpu(), predicted.it

    # 손실 및 정확도 계산
    val_loss = running_loss / total
    val_accuracy = 100 * correct / total

    return val_loss, val_accuracy, correct_examples, incorrect_examples

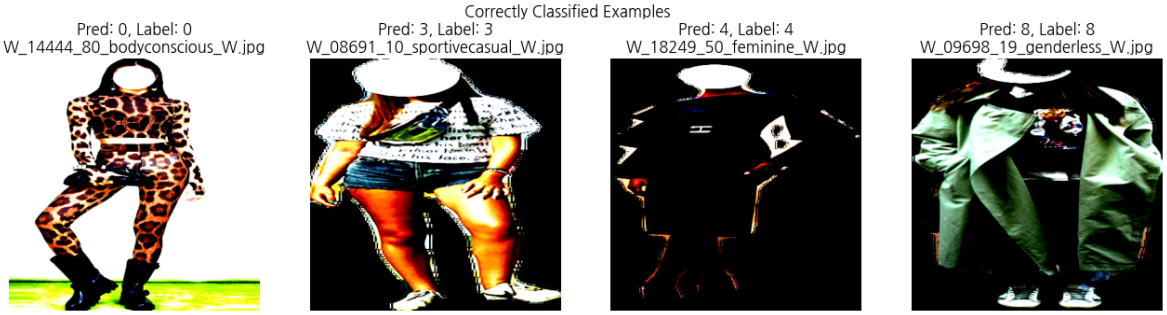
# Validation 수행 및 예제 추출
criterion = nn.CrossEntropyLoss()
_, val_accuracy, correct_examples, incorrect_examples = validate_with_exa
print(f"Validation Accuracy: {val_accuracy:.2f}%")

# 한글 폰트 설정
plt.rc('font', family='NanumGothic')

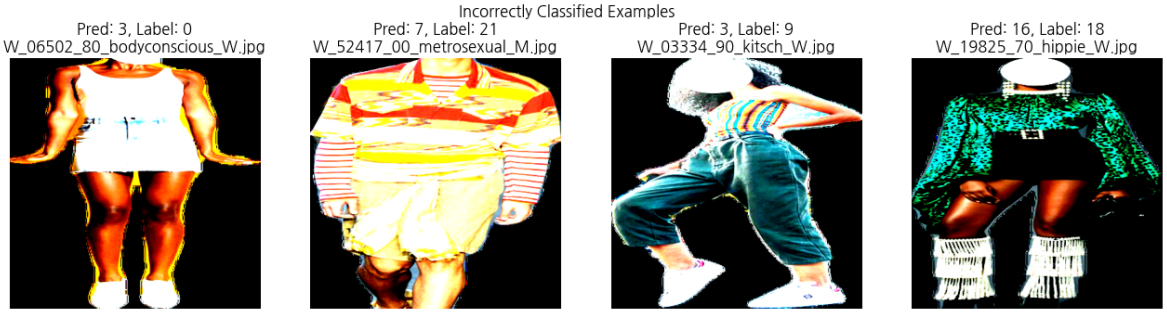
# 알맞게 분류한 예제와 잘못 분류한 예제를 출력
def plot_examples(examples, title):
    fig, axes = plt.subplots(1, 4, figsize=(16, 4))
    fig.suptitle(title)
    for i, (img, pred, label, img_path) in enumerate(examples):
        img = img.squeeze().permute(1, 2, 0).numpy()
        axes[i].imshow(img)
        axes[i].axis('off')
        axes[i].set_title(f"Pred: {pred}, Label: {label}\n{os.path.basena
    plt.show()

# 예제 시각화
plot_examples(correct_examples, "Correctly Classified Examples")
plot_examples(incorrect_examples, "Incorrectly Classified Examples")
```

```
Clipping input data to the valid range for imshow with RGB data ([0..1] fo
r floats or [0..255] for integers).
Clipping input data to the valid range for imshow with RGB data ([0..1] fo
r floats or [0..255] for integers).
Clipping input data to the valid range for imshow with RGB data ([0..1] fo
r floats or [0..255] for integers).
Clipping input data to the valid range for imshow with RGB data ([0..1] fo
r floats or [0..255] for integers).
Validation Accuracy: 64.67%
```




```
Clipping input data to the valid range for imshow with RGB data ([0..1] fo
r floats or [0..255] for integers).
Clipping input data to the valid range for imshow with RGB data ([0..1] fo
r floats or [0..255] for integers).
Clipping input data to the valid range for imshow with RGB data ([0..1] fo
r floats or [0..255] for integers).
Clipping input data to the valid range for imshow with RGB data ([0..1] fo
r floats or [0..255] for integers).
```



Mission 2-1

In [13]:
```python
# 각 파일명들을 분성하여 dataframe로 바꿔주는 함
def extract_features_from_directory(directory, extensions=['jpg', 'json']
    data = {
        "File_Name": [],
        "W/T": [],
        "ImageID": [],
        "Era": [],
        "Style": [],
        "Gender": [],
        "R_id": []
    }

    # 디렉토리 내 파일 탐색
    for filename in os.listdir(directory):
        ext = filename.split(".")[-1]
        if ext in extensions:  # 지정한 확장자 파일만 처리
            data["File_Name"].append(filename)
            parts = filename.replace("." + ext, "").split("_")

            data["W/T"].append(parts[0])
            data["ImageID"].append(parts[1])
            data["Era"].append('20' + parts[2] if int(parts[2]) < 50 else
            data["Style"].append(parts[3])
            data["Gender"].append("여성" if parts[4] == "W" else "남성")

            # json 파일일 경우 R_id 추가
            if ext == 'json':
```

```
                data["R_id"].append(parts[5])
            else:
                data["R_id"].append(None)

    return pd.DataFrame(data)
```

In [14]:
```
train_label_dir = './data/training_label'
train_image_dir = './data/training_image'
valid_label_dir = './data/validation_label'
valid_image_dir = './data/validation_image'
```

Train dataset분석

In [15]:
```
train_image_df = extract_features_from_directory(train_image_dir, 'jpg')
train_image_df.describe()
```

Out[15]:

	File_Name	W/T	ImageID	Era	Style	Gender	R_
count	4070	4070	4070	4070	4070	4070	
unique	4070	2	4066	8	27	2	
top	W_14444_80_bodyconscious_W.jpg	W	10028	2019	normcore	남성	Na
freq	1	4009	2	639	517	2248	Na

In [16]:
```
# train_image_df에서 고유한 ImageID 추출 ('.jpg' 삭제)
train_file_name = train_image_df['ImageID'].str.replace('.jpg', '', regex
print(len(train_file_name))  # 고유한 ImageID 개수 출력
```

```
4066
```

In [17]:
```
train_label_df = extract_features_from_directory(train_label_dir, 'json')
train_label_df.describe()
```

Out[17]:

	File_Name	W/T	ImageID	Era	S
count	211345	211345	211345	211345	211
unique	211345	2	70982	8	
top	W_16670_10_sportivecasual_M_141141.json	W	00016	2019	sportiveca
freq	1	181237	13	30650	23

In [18]:
```
# train_file_name을 set으로 변환하여 비교
train_file_name_set = set(train_file_name)

# train_file_name_set에 일치하는 값만 필터링
filtered_train_label_df = train_label_df[
    train_label_df['ImageID'].str.replace('.json', '', regex=False).isin(
]

# 결과 확인
print(filtered_train_label_df.head())
print(f"필터링된 데이터 개수: {len(filtered_train_label_df)}")
```

```
                                    File_Name W/T ImageID   Era           St
yle  \
12  W_64047_10_sportivecasual_W_079778.json   W   64047  2010  sportivecas
ual
30            W_07279_80_bold_M_121664.json   W   07279  1980             b
old
34         W_14869_60_minimal_W_093706.json   W   14869  1960          mini
mal
37        T_12265_00_oriental_W_228748.json   T   12265  2000        orien
tal
38         T_15967_00_ecology_W_115617.json   T   15967  2000          ecol
ogy

   Gender    R_id
12     여성  079778
30     남성  121664
34     여성  093706
37     여성  228748
38     여성  115617
필터링된 데이터 개수: 20303
```

In [19]:
```
# "성별"과 "스타일"을 기준으로 그룹화한 후, 각 그룹의 "설문ID" 개수를 세기
gender_style_stats = filtered_train_label_df.groupby(['Gender', 'Style'])

# 통계치가 담긴 데이터프레임에 컬럼명 변경 (옵션)
gender_style_stats.columns = ['Gender', 'Style', 'Survey_Count']

# Survey_Count의 총합 계산
total_survey_count = gender_style_stats['Survey_Count'].sum()

# '총합' 행 추가
total_row = pd.DataFrame({'Gender': ['총합'], 'Style': ['-'], 'Survey_Coun
gender_style_stats = pd.concat([gender_style_stats, total_row], ignore_in

# 결과 출력
print(gender_style_stats)
```

	Gender	Style	Survey_Count
0	남성	bold	1144
1	남성	hiphop	1321
2	남성	hippie	1571
3	남성	ivy	1719
4	남성	metrosexual	1226
5	남성	mods	1615
6	남성	normcore	1096
7	남성	sportivecasual	1657
8	여성	athleisure	424
9	여성	bodyconscious	500
10	여성	cityglam	329
11	여성	classic	504
12	여성	disco	141
13	여성	ecology	295
14	여성	feminine	762
15	여성	genderless	234
16	여성	grunge	122
17	여성	hiphop	243
18	여성	hippie	339
19	여성	kitsch	374
20	여성	lingerie	188
21	여성	lounge	207
22	여성	military	115
23	여성	minimal	780
24	여성	normcore	646
25	여성	oriental	369
26	여성	popart	195
27	여성	powersuit	623
28	여성	punk	237
29	여성	space	184
30	여성	sportivecasual	1143
31	총합	-	20303

Valid dataset 분석

In [22]:
```
valid_image_df = extract_features_from_directory(valid_image_dir, 'jpg')
valid_image_df.describe()
```

Out[22]:

	File_Name	W/T	ImageID	Era	Style	Gender
count	951	951	951	951	951	951
unique	951	2	951	8	27	2
top	W_14444_80_bodyconscious_W.jpg	W	14444	1950	sportivecasual	남성
freq	1	941	1	145	100	525

In [23]:
```
# valid_image_df에서 고유한 ImageID 추출
valid_image_id = valid_image_df['ImageID'].unique()
len(valid_image_id)
```

Out[23]: 951

In [24]:
```
valid_label_df = extract_features_from_directory(valid_label_dir, 'json')
valid_label_df.describe()
```

Out[24]:

	File_Name	W/T	ImageID	Era	Style	Ge
count	36382	36382	36382	36382	36382	3
unique	36382	2	12984	8	27	
top	W_08235_00_ecology_W_130653.json	W	01313	1950	sportivecasual	
freq	1	31942	12	5487	4151	2

In [25]:
```
# valid_image_id에 공통된 ImageID만 valid_label_df에서 필터링
filtered_valid_label_df = valid_label_df[valid_label_df['ImageID'].isin(v

# 공통된 ImageID만 추출한 데이터프레임 출력
filtered_valid_label_df.describe()
```

Out[25]:

	File_Name	W/T	ImageID	Era	Style	Gender
count	4262	4262	4262	4262	4262	4262
unique	4262	2	951	8	27	2
top	W_64047_10_sportivecasual_W_079778.json	W	00804	1950	ivy	남성
freq	1	4077	11	862	537	2435

In [26]:
```
# "성별"과 "스타일"을 기준으로 그룹화한 후, 각 그룹의 "설문ID" 개수를 세기
gender_style_stats = filtered_valid_label_df.groupby(['Gender', 'Style'])

# 통계치가 담긴 데이터프레임에 컬럼명 변경 (옵션)
gender_style_stats.columns = ['Gender', 'Style', 'Survey_Count']

# Survey_Count의 총합 계산
total_survey_count = gender_style_stats['Survey_Count'].sum()

# '총합' 행 추가
total_row = pd.DataFrame({'Gender': ['총합'], 'Style': ['-'], 'Survey_Coun
gender_style_stats = pd.concat([gender_style_stats, total_row], ignore_in

# 결과 출력
print(gender_style_stats)
```

	Gender	Style	Survey_Count
0	남성	bold	221
1	남성	hiphop	259
2	남성	hippie	474
3	남성	ivy	537
4	남성	metrosexual	224
5	남성	mods	438
6	남성	normcore	102
7	남성	sportivecasual	180
8	여성	athleisure	80
9	여성	bodyconscious	114
10	여성	cityglam	65
11	여성	classic	117
12	여성	disco	31
13	여성	ecology	70
14	여성	feminine	208
15	여성	genderless	24
16	여성	grunge	29
17	여성	hiphop	32
18	여성	hippie	46
19	여성	kitsch	61
20	여성	lingerie	17
21	여성	lounge	20
22	여성	military	32
23	여성	minimal	171
24	여성	normcore	50
25	여성	oriental	71
26	여성	popart	38
27	여성	powersuit	160
28	여성	punk	38
29	여성	space	75
30	여성	sportivecasual	278
31	총합	-	4262

Mission 2-2

2-1을 통해서 얻은 유효한 라벨링 data

In [27]:
```
filtered_train_label_df.describe()
```

Out[27]:

	File_Name	W/T	ImageID	Era	Sty
count	20303	20303	20303	20303	2030
unique	20303	2	4066	8	2
top	W_64047_10_sportivecasual_W_079778.json	W	01518	2010	sportivecasu
freq	1	16137	12	3224	280

In [28]:
```
filtered_valid_label_df.describe()
```

Out[28]:

	File_Name	W/T	ImageID	Era	Style	Gender
count	4262	4262	4262	4262	4262	4262
unique	4262	2	951	8	27	2
top	W_64047_10_sportivecasual_W_079778.json	W	00804	1950	ivy	남성
freq	1	4077	11	862	537	2435

In [29]:

```python
# 스타일 선호 정보를 처리하는 함수
def process_style_preference(df, json_dir, data_type='train'):
    """
    df: DataFrame containing the Filename column.
    json_dir: Directory path where the json files are stored.
    data_type: 'train' 또는 'valid'로 구분하여 처리. (default: 'train')
    """

    # 새로운 데이터프레임을 위한 딕셔너리 생성
    style_preference_data = {
        "응답자 ID": [],
        f"{data_type.capitalize()} 스타일 선호": [],
        f"{data_type.capitalize()} 스타일 비선호": []
    }

    # JSON 파일들을 탐색하고 데이터 처리
    for idx, row in df.iterrows():
        filename = row['File_Name']  # json 파일명
        filepath = os.path.join(json_dir, filename)  # 파일 경로 설정

        # 파일 존재 여부 확인 후 처리
        if os.path.exists(filepath):
            # JSON 파일 읽기
            with open(filepath, 'r') as file:
                data = json.load(file)

            # 응답자 ID 및 Q5 값 추출
            respondent_id = data['user']['R_id']
            preference = data['item']['survey']['Q5']
            img_name = data['item']['imgName']

            # 응답자 ID 추가
            style_preference_data['응답자 ID'].append(respondent_id)

            # 선호 여부에 따라 분류
            if preference == 2:  # 선호
                style_preference_data[f"{data_type.capitalize()} 스타일 선호
                style_preference_data[f"{data_type.capitalize()} 스타일 비선
            elif preference == 1:  # 비선호
                style_preference_data[f"{data_type.capitalize()} 스타일 선호
                style_preference_data[f"{data_type.capitalize()} 스타일 비선
        else:
            print(f"File {filename} does not exist.")

    # 데이터프레임으로 변환
    df_style_preference = pd.DataFrame(style_preference_data)

    return df_style_preference
```

```
# Example usage:
# train 데이터 처리
train_result_df = process_style_preference(filtered_train_label_df, train

# valid 데이터 처리
valid_result_df = process_style_preference(filtered_valid_label_df, valid

# 결과 출력
print(train_result_df)
print(valid_result_df)
```

```
       응답자 ID              Train 스타일 선호                    Train 스타일
비선호
0       58251                       None  W_64047_10_sportivecasual_W.jpg
1       63392    W_07279_80_bold_M.jpg                                None
2       60314                       None        W_14869_60_minimal_W.jpg
3       67735                       None       T_12265_00_oriental_W.jpg
4       62974  T_15967_00_ecology_W.jpg                                None
...       ...                        ...                               ...
20298   63480                       None   W_08180_80_bodyconscious_W.jpg
20299   25973                       None           W_12336_80_bold_M.jpg
20300   63738                       None       W_07759_00_cityglam_W.jpg
20301   49794    W_09760_60_mods_M.jpg                                None
20302   29960                       None            W_17603_50_ivy_M.jpg

[20303 rows x 3 columns]
      응답자 ID                     Valid 스타일 선호                     Vali
d 스타일 비선호
0      58251                                None  W_64047_10_sportivecasual_
W.jpg
1      63392           W_07279_80_bold_M.jpg
None
2      63742                                None        W_09376_00_cityglam_
W.jpg
3      64799  W_25715_10_sportivecasual_M.jpg
None
4      62349                                None          W_15141_70_hippie_
M.jpg
...      ...                                 ...
...
4257   64936                                None             W_23900_50_ivy_
M.jpg
4258   25975                                None            W_16075_80_bold_
M.jpg
4259   63359  W_01000_10_sportivecasual_W.jpg
None
4260   63207                                None          W_15499_70_hippie_
M.jpg
4261   29960                                None             W_17603_50_ivy_
M.jpg

[4262 rows x 3 columns]
```

In [30]:
```
# 유효한 train data 분석시, 응답자 ID별 총 tuple개수
len(train_result_df['응답자 ID'].unique())
```

Out[30]: 4009

In [31]:
```
# None 값을 제거하고 빈 리스트도 NaN으로 변환하는 함수
def remove_none_and_empty(lst):
```

```
    if isinstance(lst, list):
        # None을 제거한 리스트 반환, 빈 리스트일 경우 NaN으로 대체
        cleaned_list = [x for x in lst if x is not None]
        return cleaned_list if cleaned_list else np.nan  # 빈 리스트는 NaN으
    return lst
```

train dataset

In [32]:
```
# 그룹화된 데이터에서 None을 제거 (열별로 apply 사용)
grouped_train = train_result_df.groupby('응답자 ID').agg(list).reset_index

# 각 열에 대해 remove_none 함수 적용
for col in grouped_train.columns:
    if col != '응답자 ID':  # 응답자 ID 열은 처리하지 않음
        grouped_train[col] = grouped_train[col].apply(remove_none_and_emp

# 결과 출력
print(grouped_train)
```

```
      응답자 ID                                          Train 스타일 선호  \
0         12                          [W_03412_50_classic_W.jpg]
1         25  [T_01475_10_sportivecasual_M.jpg, T_14304_60_m...
2         26                           [T_09447_19_lounge_W.jpg]
3         27  [W_17260_19_normcore_M.jpg, W_16068_80_bold_M....
4         30  [W_63984_80_powersuit_W.jpg, W_18249_50_femini...
...      ...                                                ...
4004   68961                                                NaN
4005   68966                       [T_19158_19_normcore_M.jpg]
4006   68969  [T_16428_10_sportivecasual_M.jpg, T_16427_10_s...
4007   68983                       [T_19158_19_normcore_M.jpg]
4008   69016             [T_16332_10_sportivecasual_M.jpg]

                                        Train 스타일 비선호
0                                                    NaN
1                      [W_12740_00_metrosexual_M.jpg]
2                         [W_18990_50_feminine_W.jpg]
3     [W_15212_60_mods_M.jpg, W_10814_50_ivy_M.jpg, ...
4                           [W_49287_70_punk_W.jpg]
...                                                  ...
4004                     [T_19177_19_normcore_M.jpg]
4005                                                 NaN
4006  [T_16414_10_sportivecasual_M.jpg, T_16436_10_s...
4007  [T_16153_10_sportivecasual_M.jpg, T_19177_19_n...
4008  [T_16323_10_sportivecasual_M.jpg, T_16570_10_s...

[4009 rows x 3 columns]
```

In [33]:
```
# 유효한 valid data 분석시, 응답자 ID별 총 tuple개수
len(valid_result_df['응답자 ID'].unique())
```

Out[33]: 1988

valid dataset

In [34]:
```
# 그룹화된 데이터에서 None을 제거 (열별로 apply 사용)
grouped_valid = valid_result_df.groupby('응답자 ID').agg(list).reset_index

# 각 열에 대해 remove_none 함수 적용
for col in grouped_valid.columns:
```

```
    if col != '응답자 ID':  # 응답자 ID 열은 처리하지 않음
        grouped_valid[col] = grouped_valid[col].apply(remove_none_and_emp

# 결과 출력
print(grouped_valid)
```

```
      응답자 ID                                          Valid 스타일 선호  \
0         12                          [W_03412_50_classic_W.jpg]
1         27  [W_06522_50_ivy_M.jpg, W_07120_19_normcore_M.j...
2         30                         [W_18249_50_feminine_W.jpg]
3        133                                                 NaN
4        140                                                 NaN
...      ...                                                 ...
1983   68808                                                 NaN
1984   68813                                                 NaN
1985   68826                  [T_07226_10_sportivecasual_M.jpg]
1986   68883                  [T_15661_10_sportivecasual_M.jpg]
1987   68891                  [T_15440_10_sportivecasual_M.jpg]

                                      Valid 스타일 비선호
0                         [W_02651_50_feminine_W.jpg]
1                                                 NaN
2                                                 NaN
3     [W_17697_50_ivy_M.jpg, W_10073_70_hippie_M.jpg]
4                               [W_07121_80_bold_M.jpg]
...                                               ...
1983                [T_06739_10_sportivecasual_M.jpg]
1984                [T_06590_10_sportivecasual_M.jpg]
1985                                              NaN
1986                                              NaN
1987                                              NaN

[1988 rows x 3 columns]
```

In [35]:
```
# '응답자 ID'를 기준으로 inner join
merged_df = pd.merge(grouped_train, grouped_valid, on='응답자 ID', how='in

# 각 행에 대해 리스트의 원소 개수를 합산하여 'Total_Count' 컬럼 추가
merged_df['Total_Count'] = merged_df.apply(lambda row: sum(len(val) if is

# 'Total_Count' 기준으로 내림차순 정렬
merged_df = merged_df.sort_values(by='Total_Count', ascending=False)

# 결과 출력
print(merged_df)

# 'total_count' 컬럼 삭제
merged_df = merged_df.drop(columns=['Total_Count'])

# Option. 전체 결과 저장
# # CSV 파일로 저장
# file_path = './ALL_2_2.csv'
# merged_df.to_csv(file_path, index=False, encoding='utf-8-sig')
# print('경로 :', file_path, 'ALL 2-2 CSV 파일 저장완료')
```

```
      응답자 ID                                          Train 스타일 선호  \
1502   64747  [W_44330_10_sportivecasual_W.jpg, W_07894_00_c...
1052   63405  [W_01670_10_sportivecasual_M.jpg, W_06812_50_i...
1398   64346  [W_24977_70_hippie_M.jpg, W_16233_80_bold_M.jp...
1448   64561  [W_42314_70_hippie_W.jpg, W_34974_60_minimal_W...
1583   65139  [W_06684_60_mods_M.jpg, W_16523_19_normcore_M....
...      ...                                                ...
1145   63540                                              NaN
452    29256                                              NaN
1544   64928                                              NaN
453    29280                                              NaN
763    60095                                              NaN

                                          Train 스타일 비선호  \
1502  [W_34024_10_sportivecasual_W.jpg, W_48378_90_g...
1052  [W_12904_50_ivy_M.jpg, W_16501_70_hippie_M.jpg...
1398  [W_25543_60_mods_M.jpg, W_12268_80_bold_M.jpg,...
1448  [W_46417_70_military_W.jpg, W_02232_70_hippie_...
1583  [W_24517_70_hippie_M.jpg, W_57822_90_hiphop_M....
...                                                ...
1145                       [W_10207_00_oriental_W.jpg]
452                            [W_24810_80_bold_M.jpg]
1544                       [W_18171_50_feminine_W.jpg]
453                  [W_37648_10_sportivecasual_W.jpg]
763                       [W_46459_80_powersuit_W.jpg]

                                           Valid 스타일 선호  \
1502  [W_44330_10_sportivecasual_W.jpg, W_39164_00_o...
1052  [W_02879_90_hiphop_M.jpg, W_15294_50_ivy_M.jpg...
1398  [W_09154_50_ivy_M.jpg, W_24103_50_ivy_M.jpg, W...
1448  [W_38656_10_sportivecasual_W.jpg, W_06046_10_s...
1583                 [W_63644_10_sportivecasual_M.jpg]
...                                                ...
1145                                               NaN
452                           [W_28022_50_ivy_M.jpg]
1544                       [W_04927_50_feminine_W.jpg]
453                                                NaN
763                                                NaN

                                          Valid 스타일 비선호  Total_Count
1502  [W_34024_10_sportivecasual_W.jpg, W_47169_70_h...           60
1052  [W_12904_50_ivy_M.jpg, W_16501_70_hippie_M.jpg...           58
1398  [W_16430_90_hiphop_M.jpg, W_16121_80_bold_M.jp...           58
1448  [W_48457_60_minimal_W.jpg, W_22943_10_athleisu...           58
1583  [W_24517_70_hippie_M.jpg, W_54465_80_bold_M.jp...           58
...                                                ...          ...
1145                        [W_05760_60_minimal_W.jpg]            2
452                                                NaN            2
1544                                               NaN            2
453                  [W_40642_10_sportivecasual_W.jpg]            2
763                       [W_46459_80_powersuit_W.jpg]            2

[1922 rows x 6 columns]
```

In [36]:
```
# 상위 100명 추출 후 인덱스 재설정
merged_df_100 = merged_df.head(100).reset_index(drop=True)

# 결과 출력
print(merged_df_100)
```

```
# CSV 파일로 저장
file_path = './2_2.csv'
merged_df_100.to_csv(file_path, index=False, encoding='utf-8-sig')
print('경로 :', file_path, '2-2 top 100 정답 CSV 파일 저장완료')
```

```
    응답자 ID                                          Train 스타일 선호  \
0    64747  [W_44330_10_sportivecasual_W.jpg, W_07894_00_c...
1    63405  [W_01670_10_sportivecasual_M.jpg, W_06812_50_i...
2    64346  [W_24977_70_hippie_M.jpg, W_16233_80_bold_M.jp...
3    64561  [W_42314_70_hippie_W.jpg, W_34974_60_minimal_W...
4    65139  [W_06684_60_mods_M.jpg, W_16523_19_normcore_M....
..     ...                                                ...
95   63359  [W_18814_90_kitsch_W.jpg, W_03736_90_lingerie_...
96   63316  [W_01997_90_grunge_W.jpg, W_06484_19_lounge_W....
97   63207  [W_06211_10_sportivecasual_M.jpg, W_02978_10_s...
98   63569  [W_02403_70_punk_W.jpg, W_03457_60_minimal_W.j...
99   64295  [W_31375_19_normcore_M.jpg, W_01664_60_mods_M....

                                          Train 스타일 비선호  \
0   [W_34024_10_sportivecasual_W.jpg, W_48378_90_g...
1   [W_12904_50_ivy_M.jpg, W_16501_70_hippie_M.jpg...
2   [W_25543_60_mods_M.jpg, W_12268_80_bold_M.jpg,...
3   [W_46417_70_military_W.jpg, W_02232_70_hippie_...
4   [W_24517_70_hippie_M.jpg, W_57822_90_hiphop_M....
..                                                ...
95  [W_08513_00_cityglam_W.jpg, W_07785_70_hippie_...
96  [W_08115_90_hiphop_W.jpg, W_19177_50_feminine_...
97  [W_11103_00_metrosexual_M.jpg, W_09180_10_spor...
98  [W_18353_80_powersuit_W.jpg, W_08957_19_normco...
99  [W_29487_80_bold_M.jpg, W_25244_19_normcore_M....

                                           Valid 스타일 선호  \
0   [W_44330_10_sportivecasual_W.jpg, W_39164_00_o...
1   [W_02879_90_hiphop_M.jpg, W_15294_50_ivy_M.jpg...
2   [W_09154_50_ivy_M.jpg, W_24103_50_ivy_M.jpg, W...
3   [W_38656_10_sportivecasual_W.jpg, W_06046_10_s...
4                   [W_63644_10_sportivecasual_M.jpg]
..                                                ...
95  [W_00161_60_space_W.jpg, W_13518_80_powersuit_...
96  [W_06498_19_normcore_W.jpg, W_05791_10_sportiv...
97                      [W_00831_19_normcore_M.jpg]
98  [W_14785_00_cityglam_W.jpg, W_03536_70_militar...
99  [W_28563_90_hiphop_M.jpg, W_30027_50_ivy_M.jpg...

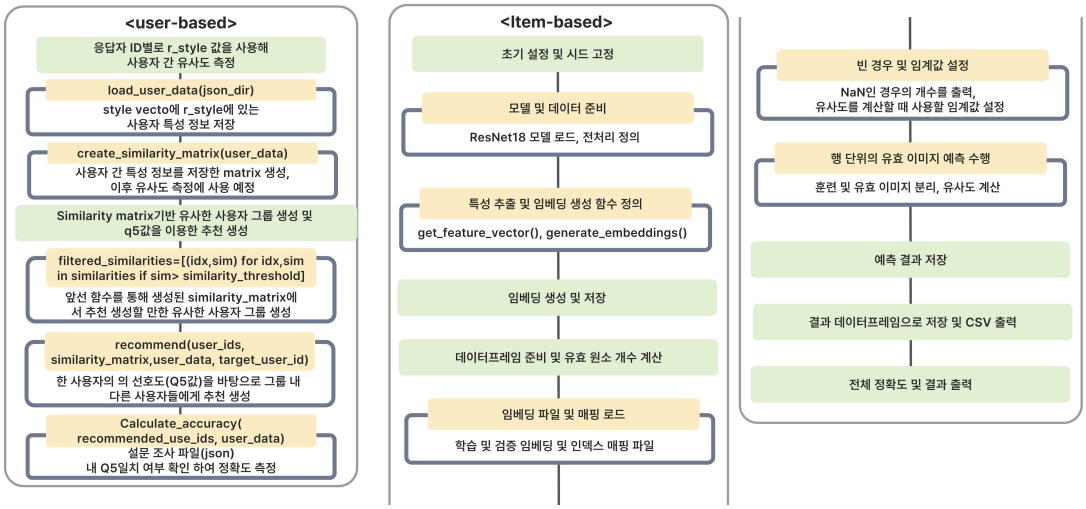
                                          Valid 스타일 비선호
0   [W_34024_10_sportivecasual_W.jpg, W_47169_70_h...
1   [W_12904_50_ivy_M.jpg, W_16501_70_hippie_M.jpg...
2   [W_16430_90_hiphop_M.jpg, W_16121_80_bold_M.jp...
3   [W_48457_60_minimal_W.jpg, W_22943_10_athleisu...
4   [W_24517_70_hippie_M.jpg, W_54465_80_bold_M.jp...
..                                                ...
95  [W_19559_00_oriental_W.jpg, W_13359_80_powersu...
96  [W_08115_90_hiphop_W.jpg, W_15071_00_cityglam_...
97  [W_12247_80_bold_M.jpg, W_06148_50_ivy_M.jpg, ...
98  [W_18353_80_powersuit_W.jpg, W_05619_00_ecolog...
99  [W_15729_90_hiphop_M.jpg, W_16374_10_sportivec...

[100 rows x 5 columns]
경로 : ./2_2.csv 2-2 top 100 정답 CSV 파일 저장완료
```

Mission 3-1

- **User-based filtering** vs **Item-based filtering**

항목	User-based filtering	Item-based filtering
개인화 수준	높은 개인화 제공	개인화 수준이 낮음
콜드 스타트 문제	새로운 사용자에게 취약	새로운 사용자에도 강함
계산 복잡도	사용자 수가 많을수록 계산 비용 증가	스타일(아이템) 수에 따라 계산 비용이 증가
추천 안정성	사용자의 선호가 변하면 실시간으로 반영됨	아이템 유사도를 미리 계산해두면 안정적인 추천 가능
사용자 데이터 필요성	많은 사용자-아이템 상호작용 데이터가 필요	아이템 간 유사도만으로도 추천 가능
새로운 아이템 추천	적응력이 높음	새로운 아이템에 대해 유사도 계산이 어려울 수 있음
적용 분야	간단한 유사성 기반 추천	이미지 유사성 분석



자세한 분석은 **PPT**에 첨부하였습니다. 참고 부탁드립니다.

Mission 3-2

In [45]:
```
from sklearn.metrics.pairwise import cosine_similarity
import matplotlib.pyplot as plt
import matplotlib.font_manager as fm
```

In [48]:
```
set_seed()  # 시드 고정
```

In [49]:
```
# 데이터셋 파라미터
TRAIN_DATA_DIR = './data/final_seg_train_image/'  # 학습 데이터 경로
VALID_DATA_DIR = './data/final_seg_valid_image'  # 검증 데이터 경로
```

In [50]:
```
# 디바이스 설정
device = "cuda" if torch.cuda.is_available() else "cpu"

# 모델 준비 및 가중치 로드
NUM_CLASSES = 31
model = models.resnet18(weights=None)
model.fc = nn.Linear(model.fc.in_features, NUM_CLASSES)
checkpoint = torch.load("/home/user/Desktop/ICT/output/best_model.pth", m
model.load_state_dict(checkpoint['model_state_dict'])
model.to(device)
model.eval()

# 새로운 전처리 정의
val_transform = transforms.Compose([
    transforms.Resize((384, 384)),
    transforms.ToTensor(),
    transforms.Normalize([0.2651, 0.2375, 0.2316], [0.2848, 0.2609, 0.255
])
```

In [51]:
```
# 특성 추출 함수 정의
# 가장 마지막 layer에서 2번째, global avg pooling 결과를 latent vector로 사용
def get_feature_vector(img_path):
    img = val_transform(Image.open(img_path)).unsqueeze(0).to(device)
    with torch.no_grad():
        features = model.conv1(img)
        features = model.bn1(features)
        features = model.relu(features)
        features = model.maxpool(features)

        features = model.layer1(features)
        features = model.layer2(features)
        features = model.layer3(features)
        features = model.layer4(features)

        features = model.avgpool(features)
        feature_vector = features.view(features.size(0), -1)  # (1, 512)
    return feature_vector.cpu().numpy().flatten()
```

In [52]:
```
# 임베딩 생성 함수
def generate_embeddings(data_dir, embedding_output_path, index_output_pat
    embeddings = []
    image_index = {}
    images = os.listdir(data_dir)

    # tqdm을 사용하여 진행 상태 표시
    for idx, img_name in enumerate(tqdm(images, desc=desc)):
        img_path = os.path.join(data_dir, img_name)
        embedding = get_feature_vector(img_path)
        embeddings.append(embedding)
        image_index[img_name] = idx  # 이미지 파일 이름과 인덱스 매핑

    # numpy 배열로 변환하여 하나의 임베딩 매트릭스 생성
    embeddings = np.vstack(embeddings)
    np.save(embedding_output_path, embeddings)
    print(f"임베딩 매트릭스가 {embedding_output_path}에 저장되었습니다.")

    # 이미지 파일 이름과 인덱스 매핑 사전을 .json 파일로 저장
    with open(index_output_path, "w", encoding="utf-8") as f:
        json.dump(image_index, f, ensure_ascii=False, indent=4)
```

```
    print(f"이미지 파일 이름과 인덱스 매핑이 {index_output_path}에 저장되었습니다."

    # 임베딩 매트릭스와 인덱스 정보 확인
    print("임베딩 매트릭스 Shape:", embeddings.shape)
    print("예시 - 첫 번째 이미지의 파일 이름:", images[0])
    print("예시 - 첫 번째 이미지의 인덱스:", image_index[images[0]])
```

In [53]:
```
# Train/valid 이미지 set 각각 임베딩 생성
generate_embeddings(TRAIN_DATA_DIR, "./train_embeddings.npy", "./train_im
generate_embeddings(VALID_DATA_DIR, "./valid_embeddings.npy", "./valid_im
```

```
Train Embeddings:   0%|          | 0/4070 [00:00<?, ?it/s]
임베딩 매트릭스가 ./train_embeddings.npy에 저장되었습니다.
이미지 파일 이름과 인덱스 매핑이 ./train_image_index.json에 저장되었습니다.
임베딩 매트릭스 Shape: (4070, 512)
예시 - 첫 번째 이미지의 파일 이름: W_14444_80_bodyconscious_W.jpg
예시 - 첫 번째 이미지의 인덱스: 0
Validation Embeddings:   0%|          | 0/951 [00:00<?, ?it/s]
임베딩 매트릭스가 ./valid_embeddings.npy에 저장되었습니다.
이미지 파일 이름과 인덱스 매핑이 ./valid_image_index.json에 저장되었습니다.
임베딩 매트릭스 Shape: (951, 512)
예시 - 첫 번째 이미지의 파일 이름: W_14444_80_bodyconscious_W.jpg
예시 - 첫 번째 이미지의 인덱스: 0
```

In [63]:
```
# 데이터 준비
file_path = "./2_2.csv"  # 실제 경로로 변경
df = pd.read_csv(file_path)

# 임베딩 및 인덱스 매핑 파일 로드
train_embeddings = np.load("./train_embeddings.npy")
valid_embeddings = np.load("./valid_embeddings.npy")
with open("./train_image_index.json", "r", encoding="utf-8") as f:
    train_index = json.load(f)
with open("./valid_image_index.json", "r", encoding="utf-8") as f:
    valid_index = json.load(f)

# 결과 저장을 위한 리스트 및 총 valid 원소 개수 카운트 변수
all_results = []
total_valid_elements = 0  # 총 valid 원소 개수를 저장할 변수
# threshold = 0.5  # 임계값 설정
```

train/valid row 무결성 재확인

In [64]:
```
# 동시에 빈 경우의 개수 확인
empty_count = df[(df['Train 스타일 선호'].isna()) & (df['Train 스타일 비선호']

print(f"'Train 스타일 선호'와 'Train 스타일 비선호'가 동시에 빈 경우의 개수: {empty_
```

```
'Train 스타일 선호'와 'Train 스타일 비선호'가 동시에 빈 경우의 개수: 0
```

In [65]:
```
# 'Valid 스타일 선호'와 'Valid 스타일 비선호'가 동시에 빈 경우의 개수 확인
empty_count = df[(df['Valid 스타일 선호'].isna()) & (df['Valid 스타일 비선호']

print(f"'Valid 스타일 선호'와 'Valid 스타일 비선호'가 동시에 빈 경우의 개수: {empty_
```

```
'Valid 스타일 선호'와 'Valid 스타일 비선호'가 동시에 빈 경우의 개수: 0
```

평가

In [66]:
```
# 결과와 관련된 변수 초기화
all_results = []
total_valid_elements = 0

# 결과 데이터를 저장할 디렉토리
output_directory = "./data"
if not os.path.exists(output_directory):
    os.makedirs(output_directory)

# 데이터 프레임 순회
for idx, row in df.iterrows():
    # train 선호/비선호, valid 선호/비선호 이미지 파일명 리스트
    train_preferred = row['Train 스타일 선호'] if pd.notna(row['Train 스타일
    train_non_preferred = row['Train 스타일 비선호'] if pd.notna(row['Train
    valid_preferred = row['Valid 스타일 선호'] if pd.notna(row['Valid 스타일
    valid_non_preferred = row['Valid 스타일 비선호'] if pd.notna(row['Valid

    # 리스트 변환
    train_preferred = train_preferred.strip("[]").replace("'", "").split(
    train_non_preferred = train_non_preferred.strip("[]").replace("'", ""
    valid_preferred = valid_preferred.strip("[]").replace("'", "").split(
    valid_non_preferred = valid_non_preferred.strip("[]").replace("'", ""

    # 각 그룹의 임베딩 리스트 생성
    train_preferred_embeddings = np.array([train_embeddings[train_index[i
    train_non_preferred_embeddings = np.array([train_embeddings[train_ind

    # Valid 이미지가 있는 경우에만 유사도 계산 및 예측 진행
    if valid_preferred or valid_non_preferred:
        row_results = []

        # valid 선호와 비선호의 모든 이미지 순회
        for img_name in valid_preferred + valid_non_preferred:
            if img_name in valid_index:  # 유효한 이미지인지 확인
                valid_embedding = valid_embeddings[valid_index[img_name]]

                # 유사도 계산
                if train_preferred_embeddings.size > 0:
                    preferred_similarities = cosine_similarity(valid_embe
                    top_preferred = np.sort(preferred_similarities)[-10:]
                else:
                    top_preferred = np.array([])

                if train_non_preferred_embeddings.size > 0:
                    non_preferred_similarities = cosine_similarity(valid_
                    top_non_preferred = np.sort(non_preferred_similaritie
                else:
                    top_non_preferred = np.array([])

                # 다수결로 선호/비선호 결정
                votes_preferred = len([sim for sim in top_preferred if si
                votes_non_preferred = len([sim for sim in top_non_preferr

                if votes_preferred > votes_non_preferred:
                    prediction = "선호"
                elif votes_preferred < votes_non_preferred:
                    prediction = "비선호"
                else:
                    prediction = "선호" if votes_preferred >= 5 else "비선호
```

```
                true_label = "선호" if img_name in valid_preferred else "ㅂ

                # 결과 저장
                row_results.append({
                    "행 인덱스": idx,
                    "이미지 이름": img_name,
                    "예측": prediction,
                    "실제 라벨": true_label
                })

                # 총 valid 원소 개수 증가
                total_valid_elements += 1
            #else:
                #print('인덱스에 없는 이미지:', img_name)

        # 현재 행에 대한 결과 추가
        all_results.extend(row_results)

# 결과를 데이터프레임으로 저장
results_df = pd.DataFrame(all_results)
results_df.to_csv(f"{output_directory}/valid_predictions_by_row.csv", ind
print("개별 행에 대한 Validation 예측 결과가 ./data/valid_predictions_by_row.c

# 전체 정확도 계산
accuracy = (results_df["예측"] == results_df["실제 라벨"]).mean()
print(f"Validation 이미지 평가 정확도: {accuracy:.2%}")

# 총 valid 원소 개수 출력
print(f"총 처리한 valid 원소 개수: {total_valid_elements}")
```

```
개별 행에 대한 Validation 예측 결과가 ./data/valid_predictions_by_row.csv에 저장
되었습니다.
Validation 이미지 평가 정확도: 71.12%
총 처리한 valid 원소 개수: 1101
```

In [ ]:

In [ ]: